

Noise, Decoherence, and Grover's Algorithm

10, 12 November • Objectives 5, 7

Two halves that belong together: the physics limiting how long a computation can run, and the first algorithm whose speedup applies to problems anyone actually has.

Modelling noise

Noise is described by **quantum channels** — completely positive trace-preserving maps built from Kraus operators:

$$\varepsilon(\rho) = \sum_k E_k \rho E_k^\dagger \quad \text{with } \sum_k E_k^\dagger E_k = I$$

The three that matter:

Amplitude damping (T1) — energy loss to the environment. Asymmetric: $|1\rangle$ decays to $|0\rangle$, never the reverse (at zero temperature).

$$E_0 = \begin{bmatrix} 1 & 0 \\ 0 & \sqrt{1-\gamma} \end{bmatrix} \quad E_1 = \begin{bmatrix} 0 & \sqrt{\gamma} \\ 0 & 0 \end{bmatrix} \quad \gamma = 1 - e^{-t/T_1}$$

Phase damping (T2) — coherence loss with no energy exchange. Populations are untouched; off-diagonal elements shrink:

$$\rho = \begin{bmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{bmatrix} \rightarrow \begin{bmatrix} \rho_{00} & \rho_{01} \cdot e^{-t/T_2} \\ \rho_{10} \cdot e^{-t/T_2} & \rho_{11} \end{bmatrix}$$

Depolarizing — the state is replaced by the maximally mixed state with probability p :

$$\varepsilon(\rho) = (1-p)\rho + p \cdot I/2$$

On the Bloch sphere, all three shrink the vector inward. Amplitude damping pulls toward the north pole; phase damping collapses the sphere toward the z-axis; depolarizing shrinks uniformly toward the center. Decoherence is the state vector leaving the surface — and points inside the ball are mixed states, which is where the picture from Week 7 pays off.

Analogy boundary — noise as a corruption rate. A per-operation error probability is the right first model, and error budgets built that way are useful. Two boundaries. First, quantum errors are **continuous** — a gate can over-rotate by 0.03 radians, which is not a discrete bit flip and has no classical counterpart. (That error correction works *anyway* is the surprising result of Week 16: syndrome measurement digitizes continuous errors.) Second, **coherent errors accumulate linearly in amplitude, hence quadratically in probability**. A thousand gates each over-rotating by the same small angle compound into a large systematic error, far worse than a thousand independent random ones — which is why randomized compiling, deliberately converting coherent error into stochastic error, is a real technique.

Building a noise model

```
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel, depolarizing_error, thermal_relaxation_error

nm = NoiseModel()
nm.add_all_qubit_quantum_error(depolarizing_error(0.001, 1), ['h', 'x', 'rz'])
nm.add_all_qubit_quantum_error(depolarizing_error(0.010, 2), ['cx'])
nm.add_all_qubit_quantum_error(
    thermal_relaxation_error(t1=200e-6, t2=120e-6, time=500e-9), ['cx'])

sim = AerSimulator(noise_model=nm)

# Or, more usefully, clone a real device's calibrated error rates:
sim = AerSimulator.from_backend(backend)
```

The second form is the one to use before spending queue time — it reproduces the device's actual per-qubit rates and reveals whether a circuit is feasible at all.

Grover's algorithm

Problem. Search N unstructured items for one satisfying $f(x) = 1$. Classically: $N/2$ expected queries. Grover: $O(\sqrt{N})$.

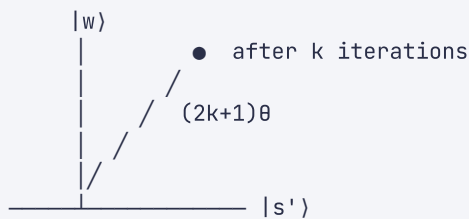
For a database of 10^6 items, 10^6 classical queries become $\sim 10^3$. Quadratic, not exponential — and unlike Deutsch–Jozsa, the speedup survives randomization. It is also **provably optimal**: no quantum algorithm does unstructured search faster (Bennett–Bernstein–Brassard–Vazirani, 1997).

The geometry. Work in the two-dimensional plane spanned by the target $|w\rangle$ and the uniform superposition of everything else $|s'\rangle$. Start in the uniform state $|s\rangle$, at angle θ from $|s'\rangle$ where $\sin \theta = 1/\sqrt{N}$.

Each Grover iteration is two reflections:

1. ORACLE: reflect about $|s'\rangle$ (flip the sign of $|w\rangle$)
2. DIFFUSION: reflect about $|s\rangle$ (invert about the mean)

Two reflections compose into a **rotation** — by 2θ toward $|w\rangle$ each time:



After k iterations the amplitude on $|w\rangle$ is $\sin((2k+1)\theta)$. Maximize it:

$$(2k+1)\theta \approx \pi/2 \quad \Rightarrow \quad k \approx (\pi/4)\sqrt{N}$$

Overshooting is real. Run too many iterations and the state rotates *past* $|w\rangle$ and success probability falls. This is genuinely counterintuitive — more work makes the answer worse — and it is a favorite exam question.

Analogy boundary — Grover as database search. The name misleads. Grover does not search a stored database; it searches the domain of a *function you can evaluate*. To search actual stored data you would first load N items into quantum superposition, which costs $O(N)$ — destroying the speedup. Grover applies where the "database" is implicit: satisfying assignments of a formula, keys of a cipher, positions in a game tree. Nothing you would put in a table.

The circuit

```
import numpy as np
from qiskit import QuantumCircuit

def grover(n, marked):
    N = 2**n
    k = int(np.floor(np.pi/4 * np.sqrt(N)))
    qc = QuantumCircuit(n, n)
    qc.h(range(n))                # uniform superposition over all N

    for _ in range(k):
        # --- oracle: phase-flip the marked state ---
        for q in range(n):
            if not (marked >> q) & 1: qc.x(q)
        qc.h(n-1); qc.mcx(list(range(n-1)), n-1); qc.h(n-1)  # multi-controlled Z
        for q in range(n):
            if not (marked >> q) & 1: qc.x(q)

        # --- diffusion: reflect about the uniform state ---
        qc.h(range(n)); qc.x(range(n))
        qc.h(n-1); qc.mcx(list(range(n-1)), n-1); qc.h(n-1)
        qc.x(range(n)); qc.h(range(n))

    qc.measure(range(n), range(n))
    return qc, k

qc, k = grover(4, marked=11)      # 16 items, target |1011>
print(f"{k} iterations for N=16")  # 3
```

```
3 iterations for N=16
{'1011': 3961, ...} ≈ 96.7% success
```

Key takeaways

- Noise is modelled by Kraus channels: amplitude damping (T1), phase damping (T2), and depolarizing. All shrink the Bloch vector inward.
- Coherent errors compound quadratically in probability; randomized compiling converts them to stochastic error deliberately.
- Grover searches in $O(\sqrt{N})$ — quadratic, provably optimal, and robust to randomization.
- Each iteration is two reflections composing to a rotation of 2θ ; run $\approx (\pi/4)\sqrt{N}$ of them.
- Overshooting reduces success probability. More iterations is not monotonically better.

Self-check

Q1 (*Objective 7*) — You run Grover with $N = 2^{10}$ and 2000 iterations instead of the optimal

1. Predict the success probability and explain the mechanism.

Q2 (*Objectives 5, 7*) — Grover on a 40-bit keyspace needs $\sim 10^6$ iterations, each with several hundred gates. Given CX error 10^{-3} , assess feasibility on current hardware and state what would change it.

Check your answers

A1 — Success probability is $\sin^2((2k+1)\theta)$ with $\sin \theta = 1/\sqrt{N} = 2^{-5}$, so $\theta \approx 0.000977$ rad.

optimal $k = 804$: $(2 \cdot 804 + 1) \cdot \theta = 1.5713 \text{ rad} \approx \pi/2 \rightarrow P \approx 1.000$
actual $k = 2000$: $(2 \cdot 2000 + 1) \cdot \theta = 3.9082 \text{ rad} \rightarrow P = \sin^2(3.9082) \approx 0.481$

Roughly **48%** — worse than half, and far worse than the optimal $\sim 100\%$.

The mechanism is that Grover is a **rotation**, not a convergence. Each iteration advances the state vector by a fixed angle 2θ in the plane spanned by $|w\rangle$ and $|s'\rangle$. There is no fixed point and no damping. Past $k = 804$ the state rotates beyond $|w\rangle$ and the amplitude on the target decreases; at 3.9082 rad it has swung well past $\pi/2$ and is heading back toward $|s'\rangle$. The process is periodic with period π/θ , so success probability oscillates indefinitely.

The practical consequence: you must know N and the number of solutions M in advance, since optimal $k = (\pi/4)\sqrt{(N/M)}$. When M is unknown, use the BBHT variant — try exponentially increasing iteration counts with randomization, which finds a solution in $O(\sqrt{(N/M)})$ expected time without prior knowledge. Amplitude amplification generalizes this further.

A2 — **Not remotely feasible**, and the gate count settles it before anything else.

$10^6 \text{ iterations} \times \sim 300 \text{ gates} = 3 \times 10^8 \text{ gates}$
Success probability $\approx (1 - 10^{-3})^{\{3 \times 10^8\}} \approx e^{\{-3 \times 10^5\}} \approx 0$

The circuit fails with certainty. Even at CX error 10^{-6} — three orders of magnitude better than any device built — you would get $e^{\{-300\}}$, still zero. Coherence time confirms it independently: at ~ 500 ns per two-qubit gate, 3×10^8 gates require ~ 150 seconds, against T_2 of $\sim 10^{-4}$ s. The computation exceeds coherence by six orders of magnitude.

What changes it is **fault-tolerant error correction** (Week 16), not better physical qubits. Below a threshold physical error rate ($\sim 10^{-3}$ for the surface code, which current hardware has reached), logical error can be suppressed exponentially by increasing code distance. The cost is overhead: roughly 10^3 – 10^4 physical qubits per logical qubit at useful distances, plus magic- state distillation for the T gates inside those multi-controlled operations.

A useful sense of scale: credible resource estimates for breaking AES-128 with Grover run to millions of physical qubits and years of runtime — which is why symmetric cryptography is considered comfortable against quantum attack, and why NIST's post-quantum standardization targeted the *asymmetric* primitives that Shor breaks exponentially. That asymmetry between Grover and Shor is the single most important fact for anyone writing a threat model, and it is Week 14's subject.